

Video title: Linear Regression in PyTorch (Module 3)

Code snippets reference

This document lists the code snippets shown in the video along with brief descriptions. We recommend reviewing this document alongside the video for the best learning experience.

Disclaimer:

The code snippets shown in this video are for conceptual illustration only. They are simplified or partial examples designed to explain the underlying ideas and may not run as-is. Please avoid copying the code directly without understanding the context and structure required to make it work.

S. No.	Code	Description
1.	<pre>import torch x = torch.tensor([[1.0],[2.0],[3.0],[4.0]]) y = torch.tensor([[2.0],[4.0],[6.0],[8.0]])</pre>	• Slide title: What is linear regression? • Purpose: Defines input-output pairs for a simple linear relationship.
2.	<pre>w = torch.randn(1, requires_grad=True) b = torch.randn(1, requires_grad=True)</pre>	• Slide title: Automatic differentiation and gradient computation in PyTorch • Purpose: Initializes model parameters w (slope) and b (bias) with gradient tracking.
3.	<pre>y_pred = w * x + b loss = ((y_pred - y) ** 2).mean()</pre>	• Slide title: Computing loss and backpropagation • Purpose: Computes the model's predictions and the mean squared error loss.
4.	<pre>learning_rate = 0.01 for epoch in range(100):</pre>	• Slide title: Gradient descent optimization • Purpose: Updates w (slope) and b (bias) using

	<pre>y_pred = w * x + b loss = ((y_pred - y) ** 2).mean() loss.backward() with torch.no_grad(): w -= learning_rate * w.grad b -= learning_rate * b.grad w.grad.zero_() b.grad.zero_()</pre>	gradients and resets them for the next iteration.
--	--	--